Lab Assignment – 2

Banker's Algorithm for Deadlock Avoidance

Course: Operating System Lab (ENCA252)
Program: BCA (AI & DS) (Research)

Task 1: System Input and Data Representation

Number of processes (5) and resources (3) are defined. Allocation matrix, Maximum matrix, and Available resources are taken as input and displayed.

```
def get_system_input():
n = int(input("Enter number of processes : "))
r = int(input("Enter number of resources : "))
# Allocation Matrix
allocation = []
for i in range(n):
row = list(map(int, input(f" Allocation for P{i}: ").split()))
allocation.append(row)
# Maximum Matrix
maximum = []
for i in range(n):
row = list(map(int, input(f" Maximum for P{i}: ").split()))
maximum.append(row)
# Available Resources
avail = list(map(int, input("Enter Available Resources: ").split()))
return n, r, allocation, maximum, avail
def display_matrices(n, r, allocation, maximum, avail):
res_headers = " " + "".join(f"R{j:<4}" for j in range(r))
print(" Allocation Matrix:")
print(res_headers)
for i in range(n):
print(f"P{i} " + "".join(f"{allocation[i][j]:<5}" for j in range(r)))
print(" Maximum Matrix:")
print(res_headers)
for i in range(n):
print(f"P{i} " + "".join(f"{maximum[i][j]:<5}" for j in range(r)))
print(" Available:", avail)
```

```
##########################################################
  Banker's Algorithm - Deadlock Avoidance
  OS Lab Assignment-2 | ENCA252
##########################################################

==========================================================
     TASK 1: System Input and Data Representation
==========================================================

Enter number of processes : 5
Enter number of resources : 3

--- Input Allocation Matrix ---
  Allocation for P0 : 0 1 0
  Allocation for P1 : 2 0 0
  Allocation for P2 : 3 0 2
  Allocation for P3 : 2 1 1
  Allocation for P4 : 0 0 2

--- Input Maximum Matrix ---
  Maximum for P0 : 7 5 3
  Maximum for P1 : 3 2 2
  Maximum for P2 : 9 0 2
  Maximum for P3 : 2 2 2
  Maximum for P4 : 4 3 3

Enter Available Resources : 3 3 2

  Allocation Matrix:
  R0   R1   R2
P0  0    1    0
P1  2    0    0
P2  3    0    2
P3  2    1    1
P4  0    0    2

  Maximum Matrix:
  R0   R1   R2
P0  7    5    3
P1  3    2    2
P2  9    0    2
P3  2    2    2
P4  4    3    3

  Available Resources:
  R0   R1   R2
   3    3    2
```

Task 2: Need Matrix Calculation

Need matrix is computed using the formula: Need[i][j] = Maximum[i][j] - Allocation[i][j]. This tells how many more resources each process may still request.

```
def calculate_need(n, r, allocation, maximum):
"""Need[i][j] = Maximum[i][j] - Allocation[i][j]"""
need = []
for i in range(n):
row = [maximum[i][j] - allocation[i][j] for j in range(r)]
need.append(row)
res_headers = " " + "".join(f"R{j:<4}" for j in range(r))
print(" Need Matrix (Need = Maximum - Allocation):")
print(res_headers)
for i in range(n):
print(f"P{i} " + "".join(f"{need[i][j]:<5}" for j in range(r)))
```

```
return need
```

```
============================================================
          TASK 2: Need Matrix Calculation
============================================================

  Need Matrix  (Need = Maximum - Allocation):
    R0   R1   R2
P0  7    4    3
P1  1    2    2
P2  6    0    0
P3  0    1    1
P4  4    3    1
```

Task 3: Banker's Safety Algorithm

Work is initialized to Available. At each step, a process whose Need <= Work is selected, its resources are released to Work, and it is marked Finish=True. This repeats until all processes finish (safe) or no process can proceed (unsafe).

```
def safety_algorithm(n, r, allocation, need, avail):
work = avail[:] # Step 1: Work = Available
finish = [False] * n # Step 2: Finish = False
safe_seq = []
count = 0
while count < n:
found = False
for i in range(n):
if not finish[i]:
# Check: Need[i] <= Work for all resources
if all(need[i][j] <= work[j] for j in range(r)):
# Simulate allocation release
work = [work[j] + allocation[i][j] for j in range(r)]
finish[i] = True
safe_seq.append(i)
count += 1
found = True
break
if not found:
break # No process can proceed — unsafe state
return finish, safe_seq
```

```
============================================================
    TASK 3: Banker's Safety Algorithm
============================================================

  Initial Work (Available) = [3, 3, 2]
  Finish array             = [False, False, False, False, False]

  Step  Process   Need               Alloc              Work After
  ---------------------------------------------------------------------
  1     P1        [1, 2, 2]          [2, 0, 0]          [5, 3, 2]
  2     P3        [0, 1, 1]          [2, 1, 1]          [7, 4, 3]
  3     P0        [7, 4, 3]          [0, 1, 0]          [7, 5, 3]
  4     P2        [6, 0, 0]          [3, 0, 2]          [10, 5, 5]
  5     P4        [4, 3, 1]          [0, 0, 2]          [10, 5, 7]
```

Task 4: Safe Sequence Determination

If all Finish[] values are True, the system is in a safe state and the safe execution sequence is displayed. If not, the system is unsafe and deadlock may occur.

```
def display_safe_sequence(finish, safe_seq, n):
if all(finish):
seq_str = " -> ".join(f"P{p}" for p in safe_seq)
print(" System is in a SAFE STATE.")
print(f" Safe Sequence: {seq_str}")
else:
print(" System is in an UNSAFE STATE.")
print(" No safe sequence — Deadlock may occur.")
```

```
============================================================
       TASK 4: Safe Sequence Determination
============================================================

  System is in a SAFE STATE.

  Safe Sequence: P1 -> P3 -> P0 -> P2 -> P4
```

Task 5: Result Analysis

The system is in a SAFE STATE. Safe sequence P1->P3->P0->P2->P4 was found. Each process in this order can acquire needed resources, execute, and release them. This matches Banker's Algorithm theory — no deadlock will occur.

```
def result_analysis(finish, safe_seq, n, need, avail):
is_safe = all(finish)
print(f" System State : {'SAFE' if is_safe else 'UNSAFE'}")
print(f" All Finish[] : {finish}")
if is_safe:
seq_str = " -> ".join(f"P{p}" for p in safe_seq)
print(f" Explanation:")
print(f" The system is in a safe state because a safe sequence")
print(f" exists: {seq_str}")
```

```
print(f" Each process can obtain all required resources,")
print(f" execute, and release them for the next process.")
print(f" Theoretical: As per Banker's Algorithm, safe sequence")
print(f" exists => no deadlock will occur.")
else:
print(" Unsafe state: No safe sequence exists.")
print(" Banker's Algorithm will deny further requests.")
```

```
============================================================
          TASK 5: Result Analysis
============================================================

  System State  : SAFE
  All Finish[]  : [True, True, True, True, True]

  Explanation:
  The system is in a safe state because a safe sequence
  exists: P1 -> P3 -> P0 -> P2 -> P4

  In this sequence, each process can eventually obtain
  all required resources, execute, and release them -
  allowing the next process in line to proceed.

  Theoretical Comparison:
  As per Banker's Algorithm theory, a system is safe if
  and only if a safe sequence exists. This result matches
  the theoretical expectation - no deadlock will occur.
```